

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

اكواد تنفيذ الدوال

اعداد : معتز عامر عبد الفتاح

MOATAZ AL ZOZ

```

#include <iostream>
#include <algorithm>
using namespace std;
// تكرار الاسترنج عدد مرات العدد داخل الائنث
string Repeat (string st , int num , string A )
{
    string r;
    for (int i = 0; i < num; i++)
    {
        r += st;
        if (i < num -1) r += A + "\n";
    }
    return r;
}

int main ()
{
    cout << Repeat ("MOATAZ" , 5 , "#");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
// عكس حروف الاسترنج
string reve (string st)
{
    string r ;
    for (int i = st.size() -1; i >= 0; i--) r += st[i];
    return r;
}

int main ()
{
    cout << reve ("MOATAZ");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
void AZ (char start , char end)
{
    for (int i = start; i <= end; i++)
    {
        cout << char(i) << " | " << i << " | " << "\n";
    }
}
int main ()
{
    AZ ('A' , 'Z');
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
    /* اصغر رقم
int minmam_num (vector <int> num)
{
    int min_num = num[0];
    for (int i : num)
        if (i < min_num) min_num = i;
    cout << min_num << "\n";
}

    /* اكبر رقم
int maxmam_num (vector <int> nums)
{
    int max_num = nums[0];
    for (int j : nums)
        if (j > max_num) max_num = j;
    cout << max_num;
}
int main ()
{
    minmam_num ({100 , 20 , 50 , -40 , 5 , 500});
    maxmam_num ({100 , 20 , 50 , -40 , 5 , 500});
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int gitNum (vector<int> num , char op = '+')
{
    int r = op == '*' ? 1 : 0;
    for (int nums : num)
    {
        switch (op)
        {
            case '+' : r += nums; break;
            case '-' : r -= nums; break;
            case '*' : r *= nums; break;
        }
    }
    return r;
}
int main ()
{
    cout << gitNum ({10 , 20 , 30 , 40 , 50} , '*');
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
// معرفة إذا كان الاسترنج بداخلها احرف ام ارقام فقط
bool isnan (string st )
{
    for (char i : st)    if (i < '0' || i > '9')    return true;
    return false;
}
int main ()
{
    cout << isnan ("55 m 55") << "\n";
    cout << isnan ("123456789");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
string trim (string st , string op , char ch)
{
    int first = 0;
    for (int i = 0; i < st.size(); i++)
    {
        if (st[i] == ch) first ++;
        else break;
    }
    int last = 0;
    for (int j = st.size() - 1; j >= 0; j--)
    {
        if (st[j] == ch) last ++;
        else break;
    }
    string R;
    if (op == "Left")
        for (int i = first ; i < st.size(); i++) R += st[i];

    else if (op == "Right")
        for (int j = 0; j < st.size() - last; j++) R += st[j];

    else if (op == "All")
        for (int e = first; e < st.size() - last; e++) R += st[e];

    else cerr << "Error: Invalid operation specified (" << op << ")\n";
    return R;
}

int main ()
{
    cout << trim ("###MOATAZ###" , "All" , '#') << "\n";
    cout << trim ("###MOATAZ###" , "Left" , '#') << "\n";
    cout << trim ("###MOATAZ###" , "Right" , '#') << "\n";
    // cout << trim ("###MOATAZ###" , "Riht" , '#') << "\n";
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
// لو عايز ارجع الحروف فقط
string clean_st (string st)
{
    string r;
    for (char ch : st)
    {
        if (ch >= 'A' && ch <= 'Z' || ch >= 'a' && ch <= 'z') r += ch;
        // if (ch >= 'A' && ch <= 'Z') r += ch;
        // else if (ch >= 'a' && ch <= 'z') r += ch;
    }
    return r;
}

int main()
{
    cout << clean_st (" ##$M5a8T5@a45Z54##");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
// لو عايز ارجع الحروف والارقام
string clean_st (string st , bool keep = true)
{
    string r;
    for (char ch : st)
    {
        if (ch >= 'A' && ch <= 'Z') r += ch;
        else if (ch >= 'a' && ch <= 'z') r += ch;
        else if (keep && ch >= '0' && ch <= '9') r += ch;
    }
    return r;
}

int main()
{
    cout << clean_st (" ##$M5a8T5@a45Z54##");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
bool firest (string st)
{
    bool allca = true;
    for (int i = 0; i < st.size(); i++)
    {
        if (i == 0 && st[i] >= 'a' && st[i] <= 'z')    return false;
        if (st[i] == '#' && st[i + 1] >= 'a' && st[i + 1] <= 'z') return false;
        if (st[i] == '@' && st[i + 1] >= 'a' && st[i + 1] <= 'z') st [i] +=
(char)32;
        if (st[i] == '@' && st[i + 1] >= 'a' && st[i + 1] <= 'z') return false;
    }
    return allca;
}
int main()
{
    cout << boolalpha;
    cout << firest ("Moataz#Amer@amer") << "\n";
    cout << firest ("Moataz#Amer@Amer") << "\n";
    cout << firest ("moataz#Amer") << "\n";
    cout << firest ("Moataz#Amer") << "\n";
    cout << firest ("Moataz#mer") << "\n";
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
bool in_array (vector <int> arr , int num)
{
    for (int i = 0; i < arr.size(); i++) if (arr.at(i) == num)    return true;
    return false;
}
int main()
{
    cout << boolalpha;
    cout << in_array ({1 , 2 , 3 , 4 , 5 , 6} , 4) << "\n";
    cout << in_array ({1 , 2 , 3 , 4 , 5 , 6} , 7);
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

string join (vector <string> ve , string st)
{
    string r;
    for (int i = 0; i < ve.size(); i++)
    {
        r += ve.at(i) ;
        if (i != ve.size() - 1)  r += st ;
    }
    return r;
}

int main()
{
    cout << join({"moataz" , "amer" , "abd" , "alftah"} , " | ");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
// جمع عناصر الفكتور على بعض
string join (vector <string> ve , string st)
{
    string r;
    for (int i = 0; i < ve.size(); i++)
    {
        r += ve.at(i) ;
        if (i != ve.size() - 1)  r += st ;
    }
    return r;
}

int main()
{
    cout << join({"moataz" , "amer" , "abd" , "alftah"} , " | ");
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
vector <string> Fillve (int num , string st)
{
    vector <string> r (num);
    for (int i = 0; i < num; i++)  r.at(i) = st;
    return r;
}
int main()
{
    vector <string> re = Fillve (10 , "Moataz");
    for (string i : re) cout << i << "\n";
    return 0;
}

```

```

#include <iostream>

#include <algorithm>
#include <vector>
using namespace std;
vector <int> numjfsjh (int start , int end)
{
    int siz = end - start + 1;
    vector <int> r ;
    for (int i = 0; i < siz; i++) r.at(i) = start + i;
    return r;
}
int main()
{
    vector <int> re = numjfsjh (5 , 15) ;
    for (int i : re) cout << i << " ";
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
string slice (string st , int start , int end , int steb = 1 , bool rev = false)
{
    string r ;
    for (int i = start; i <= end ; i += steb)
    {
        if (rev) r = st [i] + r ;
        else    r += st [i]      ;
    }
    return r ;
}
int main()
{
    cout << slice ("Moataz Amer" , 0 , 5) << "\n";      // Moataz
    cout << slice ("Moataz Amer" , 0 , 5 , 2) << "\n"; // Maa
    cout << slice ("Moataz Amer" , 0 , 5 , 1 , true); // zataoM
    return 0;
}

```

```

#include <iostream>
#include <algorithm>
using namespace std;
string format (int num)
{
    string snum = to_string(num);
    string r ;
    int count = 0;
    for (int i = snum.size() -1; i >= 0; i--)
    {
        r = snum[i] + r;
        count ++;
        if (count % 3 == 0 && i != 0) r = ',' + r;
    }
    return r ;
}
int main()
{
    cout << format (1000) << "\n";
    cout << format (10000) << "\n";
    cout << format (100000) << "\n";
    cout << format (1000000) << "\n";
    cout << format (10000000) << "\n";
    return 0;
}

```